

Final Report Template

CS 789 - Software Reengineering

Milestone 3 | Due Wednesday, December 2, 2026

Team: _____

Repository: _____

Team member name	UNLV email address

List every team member. Leave the fourth row blank for a three-person team.

The Final Report is the complete, polished version of the project report. It must grow out of the Pre-conditions and Intermediate Reports: revise earlier material in response to feedback, complete every assigned PR, remove obsolete planning language, complete the LLM-assisted work, and synthesize what the team learned across the project.

Read the complete project instructions before using this template. The project instructions define the required activities; this template structures the evidence to report at this milestone.

1. Response to Previous Feedback [TEAM]

Show how you responded to feedback from the Intermediate Report and the November 16 feedback session. Also carry forward and resolve any feedback from the Pre-conditions Report that remained open at the Intermediate milestone. The Final Report should make the evolution of the project across all three submissions visible.

Feedback source	Feedback received	Action taken	Where reflected in Final Report	Status

2. Project Context and Final Reengineering Strategy [TEAM]

- Briefly frame the project as reengineering through patch integration across diverged apache/kafka and linkedin/kafka variants.
- Present the final scope, Definition of Done, workflow, and the OORP patterns/maxims that actually guided the work.
- Summarize the final PR allocation and outcomes. Remove planning statements that are no longer relevant.

Student	PR	Category	Cherry-pick	RePatch	Manual adaptation	Tests	Final outcome

3. Project Coordination and Delivery [TEAM]

Summarize how the team coordinated across the project using the weekly meetings, shared minutes, and Discord channel. The shared minutes must remain accessible to the instructor and TAs.

- Summarize meeting attendance and whether the team maintained the required weekly cadence.
- Explain how work was tracked and whether internal target dates were met.
- Identify important blockers that were raised early and how they were resolved.
- Reflect briefly on what worked or did not work in the coordination process. Keep individual PR ownership clear.

4. Final PR Analyses [INDIVIDUAL]

Every student must complete and document all seven assigned PRs. Organize this section by PR category, and within each category identify the student responsible for each PR. Each PR analysis should tell the complete reengineering story rather than separating design, integration, and testing into different team-member roles.

4.1 Category 1 - Cherry-pick succeeds and tests are present

Repeat the PR structure below for every Category 1 PR.

PR #_____ - Analyzed by [Student Name]

- Change and repository evidence: purpose, PR URL, merge commit, changed files, and relevant history.
- Design recovery: source design, target design before integration, and the source-target design gap.
- Integration and adaptation: cherry-pick outcome and any adaptation required; show the resulting target design where appropriate.
- Testing and validation: tests executed, adequacy of the existing tests, and final result.

- Reengineering reasoning and outcome: OORP patterns/techniques used, final outcome, and the lesson from this PR.

4.2 Category 2 - Cherry-pick succeeds but tests require attention

Use the same complete PR structure. Explain why the existing tests were insufficient, what tests were added or adapted, and quantify their contribution with comparable before/after coverage where valid. Do not treat a coverage percentage alone as evidence of adequacy.

4.3 Category 3 - Cherry-pick fails but RePatch succeeds

Use the same complete PR structure. Include the Git failure, RePatch result, relevant refactorings/design evolution, any required adaptation, and testing/coverage evidence.

4.4 Category 4 - Cherry-pick and RePatch cannot resolve the conflicts

Treat these as diagnostic reengineering cases. Recover the source and target designs, explain the design/refactoring barriers, report Git and RePatch failure evidence, document any small targeted manual investigation, and explain why integration is or is not feasible. Do not perform extensive redesign merely to force success. Discuss opportunities for improving RePatch where the evidence supports them.

5. LLM-Assisted Integration and Test Generation [INDIVIDUAL]

Repeat this section for every student. Each student must complete both Part A and Part B described in the LLM-Assisted Integration and Test Generation section of the project instructions. The detailed experimental procedure is defined in the project instructions; this section reports the resulting evidence and findings.

5.1 Student: [Name]

- Part A - LLM-Assisted Patch Integration. Identify the selected Category 4 PR and report the baseline Git/RePatch failure, context supplied to the model and rationale, model/settings and prompt iterations, generated or adapted result, compilation/test verification, final outcome, and the reengineering patterns that materially influenced the experiment.
- Part B - Regression Check and LLM-Assisted Test Generation. Identify the selected successfully integrated PR and report the baseline regression/test-adequacy evidence, model/settings and prompts, tests generated and retained, reasons for discarding unsuitable tests, can-the-test-fail validation, comparable coverage or other test-adequacy evidence, and the reengineering patterns that materially influenced the experiment.
- Overall assessment: explain where the model helped, where it failed, hallucinated, or wasted effort, and what still required developer understanding and judgment. Record the exact local model/tag/digest, sampling settings, hardware, complete prompts, and raw outputs under llm-assisted/. Include failed attempts as well as successful ones.

6. Team-Level LLM Synthesis [TEAM]

Synthesize the individual LLM experiments across the team. Do not repeat each student's results. Compare the evidence to explain where local LLM assistance fits within the broader developer-in-the-loop reengineering process.

- Which kinds of Category 4 problems could or could not the models assist with?
- What common model failure modes appeared, and did compiler feedback help the model converge?

- How many generated tests were usable relative to the number generated, and why were others rejected?
- Which reengineering patterns were especially useful for evaluating LLM results?
- What remained dependent on developer understanding and judgment despite LLM assistance?

7. Cross-PR Analysis [TEAM]

Synthesize what the team learned across all assigned PRs. This section should turn the individual PR analyses into a broader investigation of cross-variant software reengineering.

Use evidence from the individual PR analyses to examine questions such as:

- What distinguished PRs that cherry-picked cleanly from those that failed?
- What kinds of source-target divergence caused integration difficulties?
- Which integration failures were associated with refactorings or design evolution?
- What kinds of failures could RePatch recover, and what remained beyond its capabilities?
- What patterns emerged in missing, inadequate, or difficult-to-adapt tests?
- Did particular files, components, or types of changes repeatedly appear in difficult cases?
- What differences emerged across Categories 1-4?

Do not simply summarize the individual PR results. Identify recurring patterns, contrasts, and lessons that help explain when and why changes can or cannot be reused across long-lived software variants.

8. Evaluation of Tools and Techniques [TEAM]

Tool / technique	Where used	What it helped reveal or accomplish	Limitations / when not useful

Discuss the techniques actually used from metrics/visualization, refactoring analysis, test coverage, software integration, and repository mining. Evaluate benefits and drawbacks rather than listing tool names.

9. Reengineering Patterns and Lessons Learned [TEAM]

- Explain which OORP patterns proved most useful and connect each to concrete project evidence.
- Discuss design recovery/adaptation, integration, testing, and refactoring lessons that generalize beyond a single PR.
- Distinguish patterns that genuinely guided decisions from patterns that merely describe the work after the fact.

- Include evidence from both the conventional PR reengineering work and the LLM-assisted experiments. Discuss cases where applying a reengineering pattern helped expose a weakness in an apparently plausible LLM result.

10. Teamwork and Project Reflection [TEAM]

- Reflect on the effectiveness of weekly meetings, Discord communication, progress tracking, and early escalation of blockers.
- Discuss how the team collaborated and shared knowledge while preserving individual ownership of assigned PRs.
- Identify what you would change if undertaking a similar reengineering project again.

11. Conclusion [TEAM]

Summarize what the project demonstrated about software reengineering in the context of cross-variant PR integration. Highlight the strongest conclusions about design evolution, integration, testing, refactoring, tool support, and developer judgment.

12. Individual Contributions

Provide a concise final mapping of each student to their seven assigned PRs and any additional team-level contributions. This section should agree with the named PR analyses and the project record.

Student	Assigned PRs	Additional team-level contributions

13. References and Supporting Evidence

Include references used in the report and links to relevant repository artifacts, shared meeting minutes, and other supporting evidence. Keep raw logs and large artifacts in the repository rather than filling the report with pages of terminal output.